

Pentesting Automation

RAFAEL COELHO

THIS PROJECT WAS BUILT WITH THE FOLLOWING TOOLS:



QEMU



KALI LINUX



APACHE
AIRFLOW



LUIGI



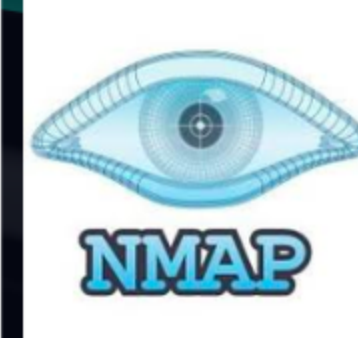
PYTHON



BASH



GO



NMAP



NIKTO



PROJECT
DISCOVERY
TOOLS

Project Discovery Tools: Nuclei | Subfinder | HTTPX | DNSX | Katana | Naabu

Python Tools: DNSReaper | Graphw00f | Arjun | SSTImap | SQLmap | SpyHunt | CORSY | XSRFProbe | XSSStrike | FavFreak

Go Tools: WayBackURLs | FFuF | Goctopus | TinjA | SURF | SmuggleFuzz | Gxss | DalFox | Subzy | MX-SPF Takeover | 403jump | Favirecon | Shortscan | Trufflehog

Other Tools: Trivy

This is a complex pentesting automation developed to aiming to participate in various bug bounty programs. It scans more than 20 types of web vulnerabilities in an automated way, saving me time from manual work so that I can focus on tasks where manual and careful analysis is necessary.

I used a Kali Linux VM (virtualizer: QEMU), and several Go, Python and Bash tools.

I started using Airflow to build this automation, but for some unknown reason, despite me adjusting several parameters, Airflow was consuming a lot of CPU processing inside the VM and was making all tasks extremely slow.

To solve this problem, I replaced Airflow with Luigi, another automation framework in Python. It not only solved my problem but also gave me more flexibility to build an automation pipeline (Fun fact: Airflow was developed by Airbnb; Luigi was developed by Spotify).

Both tools are excellent; Luigi is suitable for simpler pipelines, Airflow for robust pipelines such as Data Engineering, for example.



RunAllTasks__home_rafael_PRO__home_r2

Show task details

Show Upstream Dependencies ☐

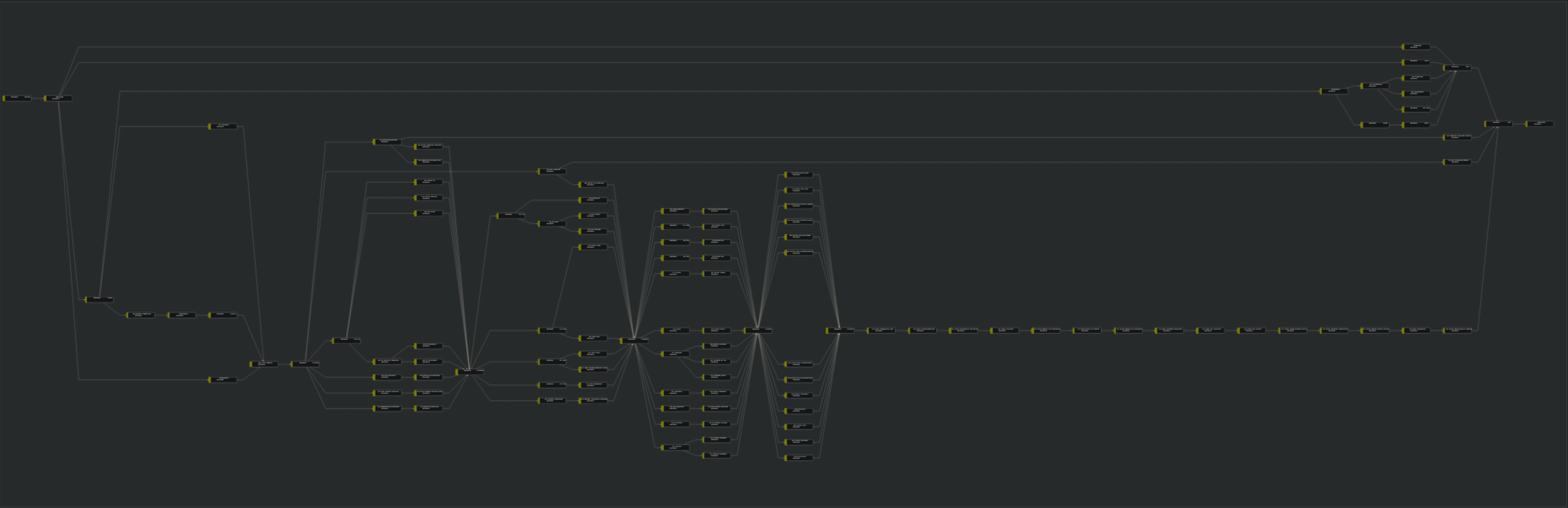
Hide Done ☐

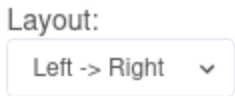
Visualisation Type

D3

SVG

Dependency Graph





LET'S WORK TOGETHER



<https://rafaelcoelho.pages.dev/>